



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Unit Convenor)
Miss Yicun TIAN (Lecturer)



Agenda

- Teaching Team
- Unit Outline
- Java Recap
- Week1 Online Content Recap



TEACHING TEAM



Convenor

Lecturer & Tutor



- **Convenor**
 - Dr Pei-Wei Tsai
ptsai@swin.edu.au
 - **Main Research Fields:**
 - Intelligent Optimisation
 - Data Analytics
 - Machine Learning
 - Differential Privacy

- **Lecturer**
 - YICUN TIAN(YI)
 - yitian@swin.edu.au
 - **Main Research Fields:**
 - Cyber Security (Phishing Attack)
 - AI (LLM & Deep learning)
 - NLP
 - Data Analytics



Yuqing Li	Tutor	-	yuqingli@swin.edu.au	9:00 – 10:00 on Wednesday
Christopher Ke	Tutor	-	cke@swin.edu.au	08:00 – 09:00 on Monday
Dr Xinyi Cai	Tutor	-	xcai@swin.edu.au	12:30 - 13:30 on Tuesday

TUTORS



ABOUT THIS UNIT (COS20028)

Pre-Requisites

[COS10022 – Introduction to Data Science]

AND

[COS20007 – Object-Oriented Programming or COS30016 – Programming in Java]



Learning and Teaching Structure

Learning and Teaching Structure

Category	Activity	Total Hours	Hours per Week	Teaching Period Weeks
Live Online	Lectures	24 hours	2 hours	Weeks 1 to 12
On Campus	Class	24 hours	2 hours	Weeks 1 to 12
Online	Directed Online Learning and Independent Learning	12 hours	1 hour	Weeks 1 to 12
Unspecified Activities	Independent Learning	90 hours	7.5 hours	Weeks 1 to 12





Cloudera VM

Please Download the VM: [COS20028 VM](#) ↗ (Windows User), [COS20028 VM qcow2](#) ↗ (Mac User)

We'll operate the Big Data environment on this virtual machine.

If you are using a Windows system, you can use the VMware Workstation Player: [VMware-workstation-full-17.6.4-24832109.exe](#) ↓

For Mac users (with Intel processors), you should use the VMware Fusion: [VMware-Fusion-13.6.4-24832108 universal.dmg](#) ↓

, or use this VirtualBox version: [VirtualBox-7.0.8 BETA4-156879-macOSArm64.dmg](#) ↓

For Mac Users with M1, M2 or M3, you may need to use UTM. Please check the following video.

https://www.youtube.com/watch?v=1suVXymrD0Q&ab_channel=SYSADMIN102%E2%84%A2 ↗

Lab Environment

Hardware

A PC or a laptop.

acle
ox
tional)

Assessment and Task Details

Assessment Task	Task Type	Weighting	Submission Due Date
Assignment 1	Individual	20%	End of Week 05
Assignment 2		20%	End of Week 11
Online Test 1		15%	Week 04 Class
Online Test 2		15%	Week 07 Class
Online Test 3		30%	Week 12 Class





**DEADLINE
* EXTENDED ***

Special Consideration?

If you encounter some difficulties such as medical issues, which affects your progress, during the study, you'll need to launch a special consideration.

The special consideration will only extend the deadline in a reasonable range for submitting the assignments. It will not be used to twist the evaluation criteria.

A valid medical certificate should be issued by any authorised Australian GP and clearly state the patient, and the impact period of unfit for study.



**DEADLINE
* EXTENDED ***

Special Consideration?

A special exam is offered to students who are granted special considerations, unless it is possible to accommodate the special considerations during the semester.



Before the end of the semester: deadline extended.
After the end of the semester: a special exam is required.

Week 1 Overview: What You'll Learn

Source	Content
Online Videos (Canvas)	Part 1: Unit Overview Part 2: The Platform for Storing Big Data and Big Data Ecosystem Part 3: HDFS Principle and Architecture
Live Lecture	Unit Overview Java Recap & Practices Online Content Overview
Lab Activities	VM Setup Hadoop Environment Preparation Hadoop Commands Practice



JAVA RECAP

Basic Concept – Test (T/F)

- Variables and Data Types

In Java, variables must be declared with a data type before they can be used.

- Operators

The == operator in Java is used to compare two String objects for equality.

- Conditional Statements

An if statement can be written without an else block.

- For Loop

A for loop in Java always runs at least once, regardless of the condition.

- While Loop

The condition in a while loop is checked before each iteration.

- Methods (Functions)

A method in Java can only return values of type int or double.

- Classes and Objects (Basics)

An object must be created using the new keyword before its methods can be called.

Arrays

Arrays in Java can store multiple values of the same type, and each element is accessed using an index starting at 1.

Basic Concept

Concept	Description	Simple Example
Variables and Data Types	Define variables and store data	<code>int age = 20; String name = "Tom";</code>
Operators	Arithmetic, comparison, and logical operations	<code>int sum = 5 + 3; boolean bigger = sum > 5;</code>
Conditional Statements	Execute different code based on conditions	<code>if (age >= 18) { System.out.println("Adult"); }</code>
for Loop	Repeat a task a specified number of times	<code>for (int i = 0; i < 3; i++) { System.out.println(i); }</code>
while Loop	Repeat while a condition is true	<code>int n = 0; while (n < 3) { n++; }</code>
Methods (Functions)	Encapsulate code blocks with return values or actions	<code>public static int add(int a, int b) { return a+b; }</code>
Classes and Objects (Basics)	Classes are templates, objects are instances	<code>Dog d = new Dog(); d.name="Buddy"; d.bark();</code>
Arrays	Store multiple elements of the same type	<code>int[] nums = {1, 2, 3};</code>
Basic Output	Print information to the console	<code>System.out.println("Hello World!");</code>

1. Java Program Structure

- A Java program is made of **classes** and **methods**.
- Every program needs a **main() method** as the entry point.
- The **class name must match the file name**.

```
java

public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

2. Variables and Data Types

- **Primitive types:** int, double, boolean, char
- **Reference types:** String, objects
- **Declaration and initialization:**

```
// Primitive types
int age = 25;           // integer
double price = 19.99;  // floating-point number
boolean isStudent = true; // true or false
char grade = 'A';      // single character
```

```
// Reference types
String city = "Melbourne"; // text string

// Object example
Dog myDog = new Dog();      // create a Dog object
myDog.name = "Buddy";
myDog.bark();
```

```
class Dog {
    String name;

    void bark() {
        System.out.println(name + " says Woof!");
    }
}
```


3. Operators

- Arithmetic Operators

+, -, *, /, %

Used for basic math operations.

- Comparison Operators

>, <, ==, !=

Used to compare two values and return true or false.

- Logical Operators

&&, ||, !

Used to combine or reverse boolean values.

java

```
int a = 5, b = 3;  
System.out.println(a + b);    // 8  
System.out.println(a > b);    // true  
System.out.println(a > 0 && b > 0); // true
```

4. Conditional Statements

1) if Statement

Runs code **only if** a condition is true.

2) if-else Statement

Runs one block if true, **another if false**.

3) switch-case Statement

Selects a block to run based on a value.

```
int day = 3;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    default: System.out.println("Other");
}
```

java

```
if (score >= 60) {
    System.out.println("Pass");
}
```

```
if (score >= 60) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

5. Loops – for Loop

- **What is a for Loop?**
- **Used when the number of repetitions is known.**
- **It has initialization, condition, and update.**

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Count: " + i);  
}
```

```
Count: 0  
Count: 1  
Count: 2
```

6. Loops – while & do-while Loop

- **while Loop (Condition Loop)**
- **Runs while the condition is true.**
- **do-while Loop (Runs at Least Once)**

```
int n = 0;
while (n < 3) {
    System.out.println("n = " + n);
    n++;
}
```

```
int n = 0;
do {
    System.out.println("n = " + n);
    n++;
} while (n < 3);
```


7. Array

What is an Array?

- An array stores multiple values of the same type in a single variable.
- Each element is accessed by its index (starting at 0).
- Example: Declare and Initialize
 - `int[] nums = {1, 2, 3};`
- Example: Loop Through an Array
 - `nums[0] = 1`
 - `nums[1] = 2`
 - `nums[2] = 3`

```
for (int i = 0; i < nums.length; i++) {  
    System.out.println(nums[i]);  
}
```

8. Methods (Functions)

What is a Method?

- A **method** is a block of code that performs a task.
- It can **take inputs (parameters)** and **return a value**.
- **Example: Method with Return Value**

```
public static int add(int a, int b) {  
    return a + b;  
}
```

- **Example : Method with void (No Return Value)**

```
public static void greet(String name) {  
    System.out.println("Hello " + name);  
}
```

9. Classes and Objects

Why Learn Classes and Objects?

- **Classes** are blueprints for creating objects.
- **Objects** are instances of classes, representing real-world entities.
- They help **organize code, reuse functionality, and model real-world behavior.**

9. Classes and Objects

```
// We need separate variables for each cat
```

```
String cat1Name = "Kitty";
```

```
int cat1Age = 2;
```

```
String cat2Name = "Mimi";
```

```
int cat2Age = 1;
```

```
// A function to print cat information
```

```
void printCat(String name, int age) {  
    System.out.println(name + " is " + age + " years old.");  
}
```

```
// A function to make the cat meow
```

```
void meow(String name) {  
    System.out.println(name + " says meow!");  
}
```

Without Class and Object

- We have to create many separate variables for each cat.
- We always need to pass all data into functions manually.
- If we want to add a new property (e.g., color), we must change every function.

```
// We must pass all data every time we call the functions
```

```
printCat(cat1Name, cat1Age);  
meow(cat1Name);
```

```
printCat(cat2Name, cat2Age);  
meow(cat2Name);
```

9. Classes and Objects

```
// Define a Cat class to store data and behavior together
class Cat {
    String name;
    int age;

    // Constructor to create a cat with name and age
    Cat(String n, int a) {
        name = n;
        age = a;
    }

    void printInfo() {
        System.out.println(name + " is " + age + " years old.");
    }

    void meow() {
        System.out.println(name + " says meow!");
    }
}
```

With Class and Object

- ✓ Data (name, age) and actions (printInfo, meow) are combined.
- ✓ Adding a new property (e.g., color) only requires modifying the Cat class once.
- ✓ Creating more cats is easy: `new Cat("Tom", 3);`

```
// Create two Cat objects
Cat cat1 = new Cat("Kitty", 2);
Cat cat2 = new Cat("Mimi", 1);

// We can directly call methods without passing data every time
cat1.printInfo(); // Kitty is 2 years old.
cat1.meow();      // Kitty says meow!

cat2.printInfo(); // Mimi is 1 years old.
cat2.meow();      // Mimi says meow!
```


Basic Concept - Test

- **Variables and Data Types**

In Java, variables must be declared with a data type before they can be used.

 **True**

- **Operators**

The **==** operator in Java is used to compare two String objects for equality.

 **False** (It compares references; `.equals()` should be used for content comparison.)

- **Conditional Statements**

An if statement can be written without an else block.

 **True**

- **for Loop**

A for loop in Java always runs at least once, regardless of the condition.

 **False** (This is true for do-while, not for.

Basic Concept - Test

while Loop

The condition in a while **loop** is checked before each iteration.

 **True**

Methods (Functions)

A method in Java can only return values of type int **or** double.

 **False** (Methods can return any data type or void.)

Classes and Objects (Basics)

An object must be created using the new **keyword** before its methods can be called.

 **True**

Arrays

Arrays in Java can store multiple values of the same type, and each element is accessed using an index starting at 1.

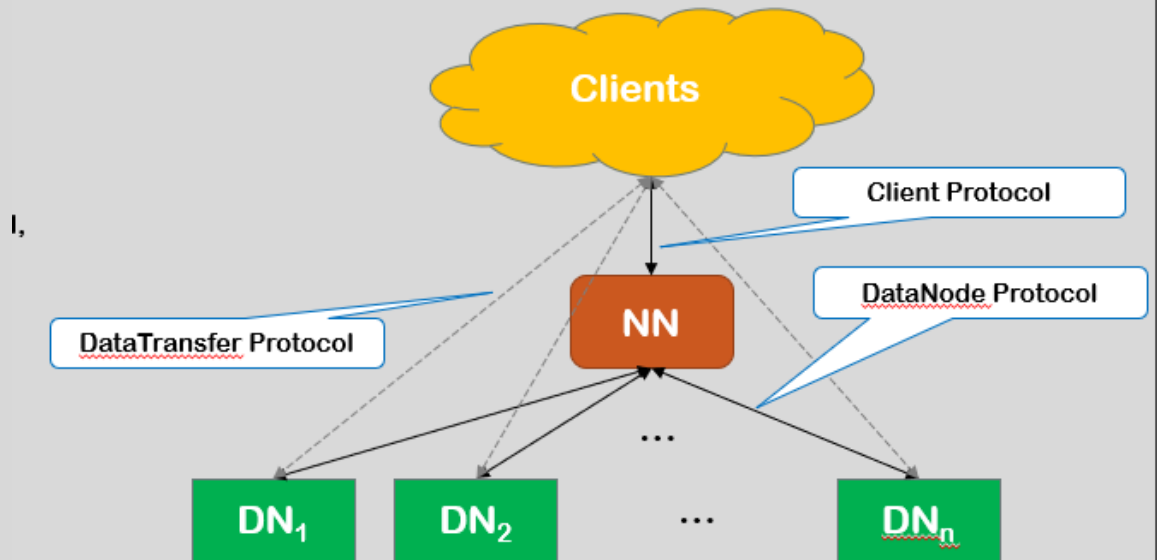
 **False** (In Java, array indices start at 0, not 1.)



ONLINE CONTENT RECAP

Hadoop – T/F Question

1. In HDFS, the Namenode is the master that manages metadata and the Datanodes are slaves that store actual data.
2. The Namenode stores both the metadata and the actual file data.
3. Datanodes are responsible for storing data blocks and serving read/write requests from clients.
4. The Namenode receives heartbeat signals from all Datanodes to check their status.



Hadoop – T/F Question

1. In HDFS, the Namenode is the master that manages metadata and the Datanodes are slaves that store actual data.

☒ True

2. The Namenode stores both the metadata and the actual file data.

☒ False (The Namenode stores only metadata, not the actual file data)

3. Datanodes are responsible for storing data blocks and serving read/write requests from clients.

☒ True

4. The Namenode receives heartbeat signals from all Datanodes to check their status.

☒ True

HDFS – T/F Question

- 1) In HDFS, each file is stored as multiple blocks.
- 2) The default block size in Apache Hadoop 2.x is 64 MB.
- 3) A 200 MB file will be stored as two blocks in HDFS if the block size is 128 MB.
- 4) HDFS splits files into blocks to support parallel processing and fault tolerance.
- 5) Each block in HDFS must always be fully filled with data.



HDFS – T/F Question

In HDFS, each file is stored as multiple blocks.

 True

The default block size in Apache Hadoop 2.x is 64 MB.

 False (The correct answer is 128 MB)

A 200 MB file will be stored as two blocks in HDFS if the block size is 128 MB.

 True

HDFS splits files into blocks to support parallel processing and fault tolerance.

 True

Each block in HDFS must always be fully filled with data.

 False (The last block can be partially filled, e.g., 72 MB)

HDFS Block Replication – T/F Question

1. HDFS stores multiple replicas of each block to improve fault tolerance.
2. By default, HDFS keeps two replicas of each block.
3. If one DataNode fails, HDFS can still access the file using replicas stored on other DataNodes.
4. All replicas of a block are always stored on the same rack to reduce network traffic.
5. The Namenode keeps track of where each block replica is stored.



HDFS Block Replication – T/F Question

1. HDFS stores multiple replicas of each block to improve fault tolerance.

 True

2. By default, HDFS keeps two replicas of each block.

 False (The default is 3 replicas.)

3. If one DataNode fails, HDFS can still access the file using replicas stored on other DataNodes.

 True

4. All replicas of a block are always stored on the same rack to reduce network traffic.

 False (Replicas are placed on different racks to improve fault tolerance.)

5. The Namenode keeps track of where each block replica is stored.

 True

Overview

1. HDFS Architecture
 - Master-Slave Architecture
 - Role of Namenode
 - Role of Datanode
2. HDFS Blocks Files
3. Block Replication
4. Data Read/Write Process
 - Write Process
 - Read Process
5. Design Goals of HDFS
 - Scalable storage
 - Fault tolerance
 - Support for parallel processing